MediCore Hospital Management System

Database Systems Laboratory (CSE 3110) Project Report

---

1. Problem Statement & Core Objectives

Problem Identified: Traditional healthcare facilities rely heavily on fragmented, paper-based workflows for patient intake, scheduling, laboratory diagnostics, and financial accounting. This manual approach leads to significant operational bottlenecks, including long patient wait times, misplaced medical records, charting errors, and opaque revenue tracking—making it exceptionally difficult to securely manage and audit physician commission payouts.

Core Objective: The objective of MediCore is to engineer a secure, centralized, and fully digitized Hospital Management System (HMS). By leveraging a highly normalized Oracle 11g relational database backend, MediCore aims to automate dynamic appointment queues, ensure instantaneous and transparent financial ledger updates, and provide distinct, role-based dashboards that seamlessly connect patients, doctors, lab technicians, and hospital administrators.

2. Executive Summary & Purpose

MediCore is a comprehensive, full-stack Hospital Management System (HMS) developed to digitize, automate, and streamline the core operational workflows of a modern healthcare facility. The primary purpose of this detailed report is to serve as the definitive technical reference for the application's robust database architecture, detailing exactly how data traverses from the React frontend, through the Node.js API layer, and into the highly structured Oracle 11g database.

Global System Architecture:

- **Presentation Tier (Frontend):** Next.js (React) and Tailwind CSS. Manages local client state, handles routing, and strictly protects views based on user roles (Admin, Doctor, Patient, Lab).
- **Application Tier (Backend):** Node.js and Express.js. Validates JWT tokens for authentication, enforces business logic (like calculating commission splits), and executes backend SQL queries via the `oracledb` driver.
- **Data Tier (Database):** Oracle Database 11g. Provides persistent, ACID-compliant data storage utilizing sequences, triggers, relational constraints, and Large Objects (CLOB).

2. Database Schema & Data Dictionary

The MediCore database is highly normalized, consisting of 9 core tables designed to handle the hospital's workflows efficiently without data redundancy.

Table Name	Core Purpose & Key Attributes
USER_ACCOUNT	Gateway for system authentication. Stores `Username`, bcrypt `Password_Hash`, and authorization `Role`. Links to either Patient or Doctor entities.
PATIENT	Stores demographic and contact information. Key fields include `Blood_Group`, `Phone`, and `Emergency_Contact`.
DOCTOR	Stores physician details and links to departments. Crucially tracks the `Consultation_Fee` for automated financial ledger split calculations.
DEPARTMENT	Lookup table categorizing doctors (e.g., General Medicine, Cardiology) containing the `Department_Name`.
APPOINTMENT	The bridge between patients and doctors. Tracks the `Appointment_Date`, dynamic `Queue_Number`, and visit `Status`.
PRESCRIPTION	The medical output of an appointment. Uses Oracle CLOBs to store extensive, unstructured `Medicines` and `Notes`.
LAB_TEST	The hospital's catalog of available diagnostic procedures and their associated `Test_Fee`.
LAB_TEST_RECORD	Tracks a specific instance of a patient taking a test. Includes `Payment_Status`, findings in `Result_Details`, and a `Waive_Commission` flag for charity.
FINANCIAL_LEDGER	The single source of truth for revenue. Logs `Total_Amount`, auto-calculated `Doctor_Amount`, `Admin_Amount`, and an `Is_Cleared` payout tracker.

Relational Database Schema Architecture

The schema diagram below illustrates the exact structural foundation of the MediCore database. It highlights the 9 core normalized tables, their primary keys, foreign key constraints, and the relationships that ensure strict data integrity across all hospital modules.

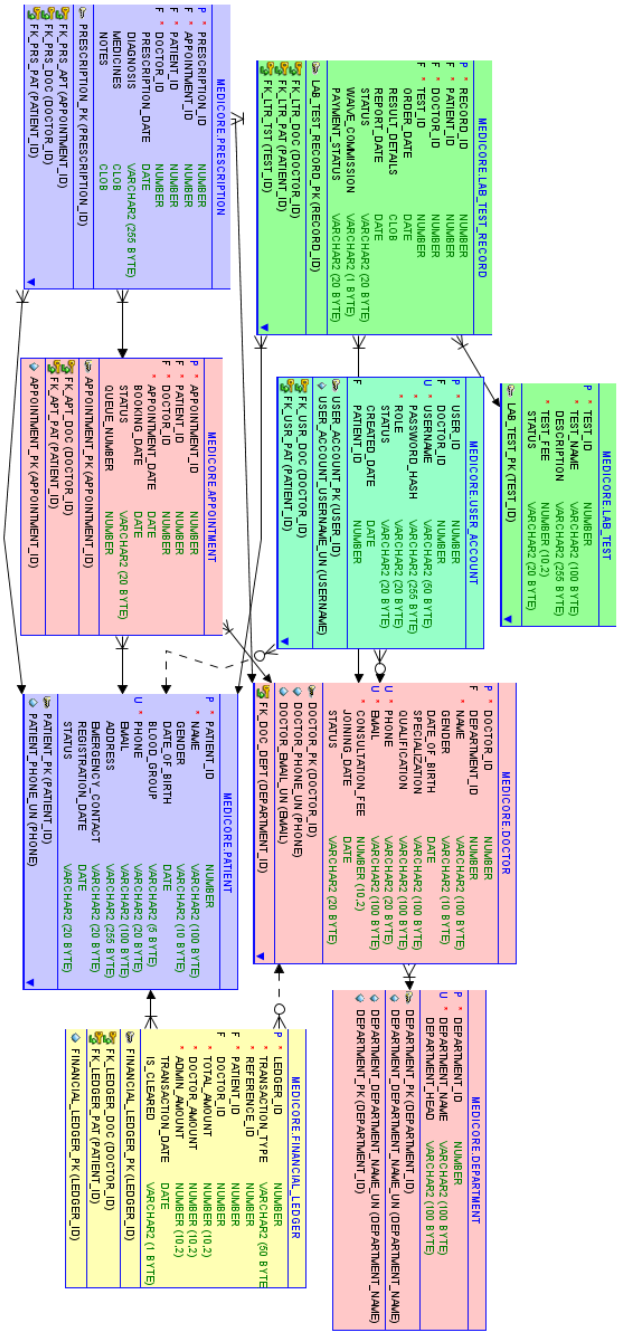
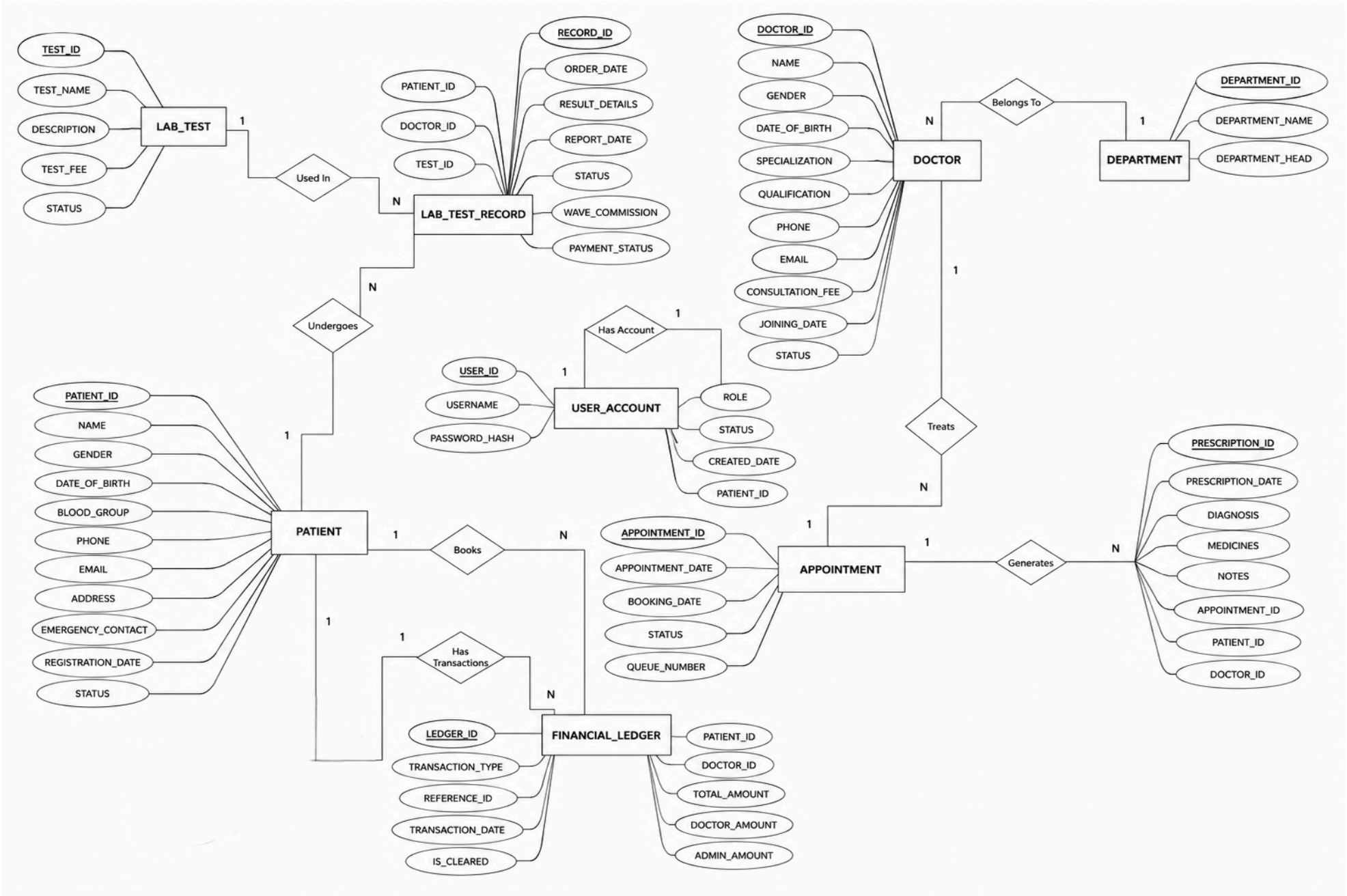


Figure 1: Oracle 11g Relational Schema depicting Primary and Foreign Key constraints.


LAB_TEST
TEST_ID
TEST_NAME
DESCRIPTION
TEST_FEE
STATUS
Used In
1
N
LAB_TEST_RECORD
RECORD_ID
ORDER_DATE
RESULT_DETAILS
REPORT_DATE
STATUS
WAVE_COMMISSION
PAYMENT_STATUS
PATIENT_ID
DOCTOR_ID
TEST_ID
Undergoes
N
1
DOCTOR
DOCTOR_ID
NAME
GENDER
DATE_OF_BIRTH
SPECIALIZATION
QUALIFICATION
PHONE
EMAIL
CONSULTATION_FEE
JOINING_DATE
STATUS
Belongs To
N
1
DEPARTMENT
DEPARTMENT_ID
DEPARTMENT_NAME
DEPARTMENT_HEAD
Has Account
1
1
USER_ACCOUNT
USER_ID
USERNAME
PASSWORD_HASH
ROLE
STATUS
CREATED_DATE
PATIENT_ID
PATIENT
PATIENT_ID
NAME
GENDER
DATE_OF_BIRTH
BLOOD_GROUP
PHONE
EMAIL
ADDRESS
EMERGENCY_CONTACT
REGISTRATION_DATE
STATUS
1
Books
1
N
Treats
1
N
APPOINTMENT
APPOINTMENT_ID
APPOINTMENT_DATE
BOOKING_DATE
STATUS
QUEUE_NUMBER
1
Generates
N
PRESCRIPTION
PRESCRIPTION_ID
PRESCRIPTION_DATE
DIAGNOSIS
MEDICINES
NOTES
APPOINTMENT_ID
PATIENT_ID
DOCTOR_ID
1
Has Transactions
N
FINANCIAL_LEDGER
LEDGER_ID
TRANSACTION_TYPE
REFERENCE_ID
TRANSACTION_DATE
IS_CLEARED
PATIENT_ID
DOCTOR_ID
TOTAL_AMOUNT
DOCTOR_AMOUNT
ADMIN_AMOUNT

3. PL/SQL Constructs: Sequences & Triggers

Because Oracle Database 11g does not natively support `AUTO_INCREMENT` columns, MediCore utilizes **Sequences** paired with **Before Insert Triggers** to guarantee mathematically unique primary keys, preventing race conditions under high concurrent loads.

Example: The Appointment Table

```
-- 1. The Sequence
CREATE SEQUENCE medicore.apt_seq START WITH 1 NOCACHE ORDER;

-- 2. The Trigger
CREATE OR REPLACE TRIGGER medicore.trg_apt_id BEFORE
    INSERT ON medicore.appointment
    FOR EACH ROW
    WHEN ( new.appointment_id IS NULL )
BEGIN
    :new.appointment_id := medicore.apt_seq.nextval;
END;
/
```

This identical PL/SQL pattern is replicated across all core entities within the database.

4. Detailed Database Workflows

4.1 Appointment Booking & Queue Calculation

To ensure patients are seen in an orderly fashion, the system assigns a dynamic queue number for the specific day. The backend executes an aggregate SQL query to find the highest existing queue number for that doctor on the requested date:

```
SELECT NVL(MAX(Queue_Number), 0) + 1 AS Next_Queue
FROM APPOINTMENT
WHERE Doctor_ID = :doctorId
AND TRUNC(Appointment_Date) = TO_DATE(:date, 'YYYY-MM-DD')
```

4.2 Immediate Financial Ledger Updates

Because MediCore operates on a prepaid consultation model, the financial ledger is updated in the exact same transaction block as the appointment booking or test order to guarantee atomicity.

```
INSERT INTO FINANCIAL_LEDGER
(Transaction_Type, Reference_ID, Patient_ID, Doctor_ID, Total_Amount, Doctor_Amount
Admin_Amount)
VALUES ('Appointment', :aptId, :patientId, :doctorId, :total, :docCut, :adminCut)
```

4.3 Hospital Revenue Analytics

The system provides a bird's-eye view of profitability by executing aggregate SQL functions directly on the ledger, parsing consultation versus laboratory revenue instantly:

```
SELECT
    SUM(CASE WHEN Transaction_Type = 'Appointment' THEN Admin_Amount ELSE 0 END) AS
    SUM(CASE WHEN Transaction_Type = 'Lab Test' THEN Admin_Amount ELSE 0 END) AS To
    SUM(Admin_Amount) AS Grand_Total
FROM FINANCIAL_LEDGER
```

5. Security Considerations

- **SQL Injection Prevention:** The backend strictly uses Oracle Bind Variables (e.g., `:patientId`, `:password`) via the `oracledb` driver, making standard SQL injection mathematically impossible.
- **Password Storage:** No plaintext passwords exist anywhere. The `bcrypt` hashing algorithm employs a variable salt, ensuring that even identical passwords result in different hashes to thwart rainbow table attacks.
- **Cross-Site Scripting (XSS):** The React frontend framework natively escapes string variables rendered in the DOM, preventing script injection.

6. System Dashboards & User Interfaces

The following screenshots demonstrate the functional deployment of the MediCore system across its various access roles, highlighting the distinct data presentations mapped to specific user authorizations.

Administrator Portal

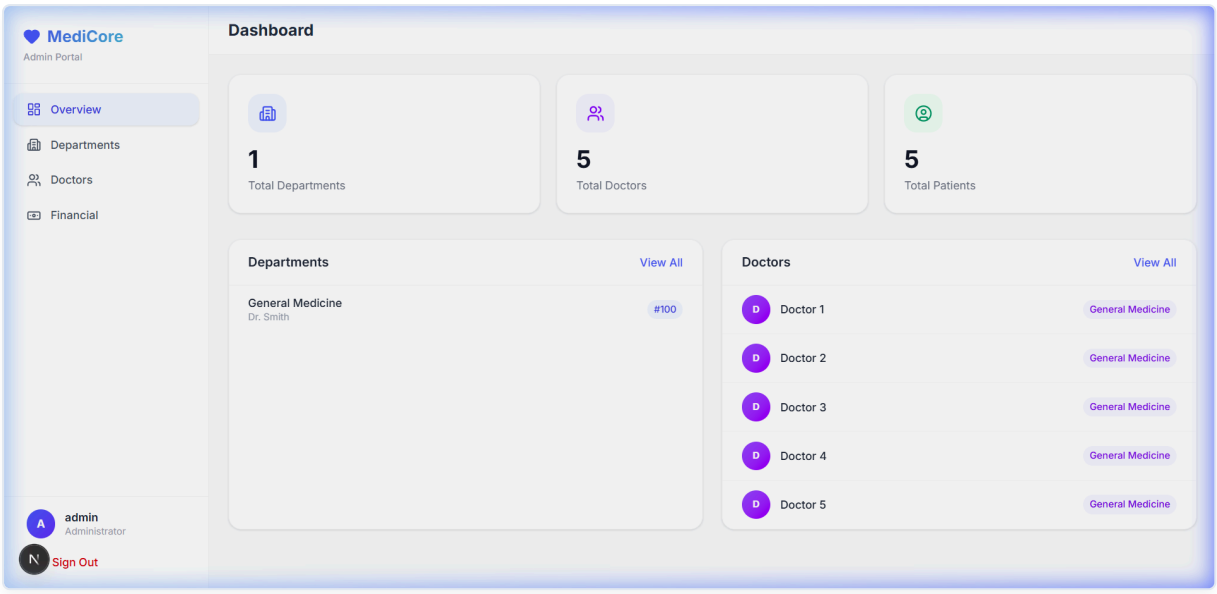


Figure 3: Admin Dashboard displaying system overview metrics, registered doctors, and department statistics.

Patient Dashboard

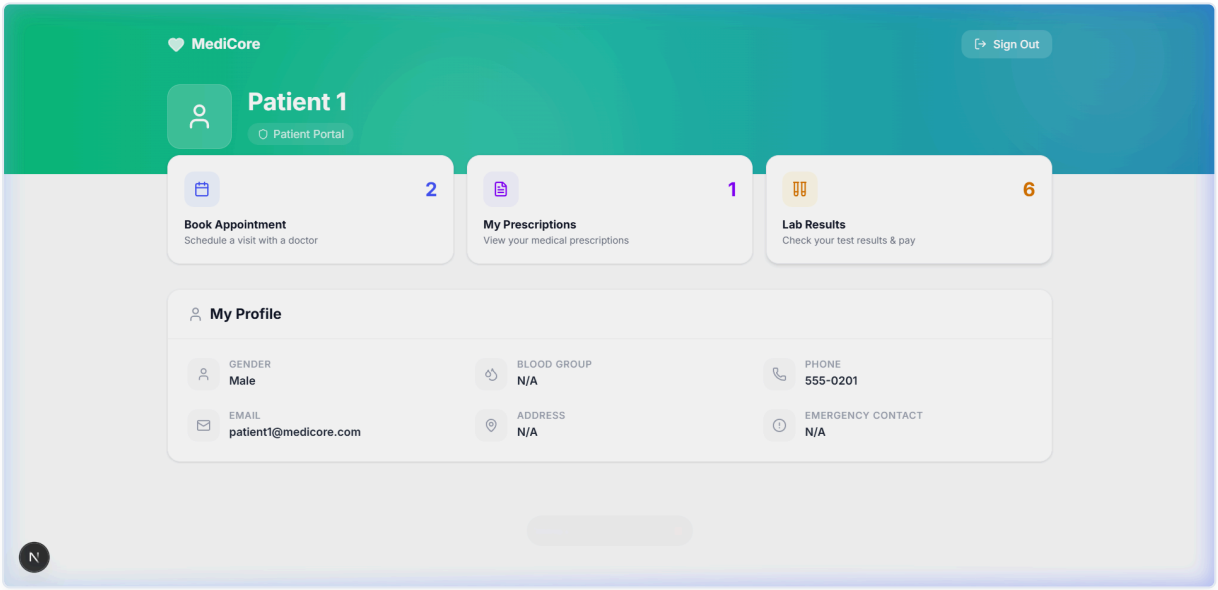


Figure 4: The Main Patient Portal offering routing to appointment bookings, prescriptions, and lab records.

Patient Lab Results Module

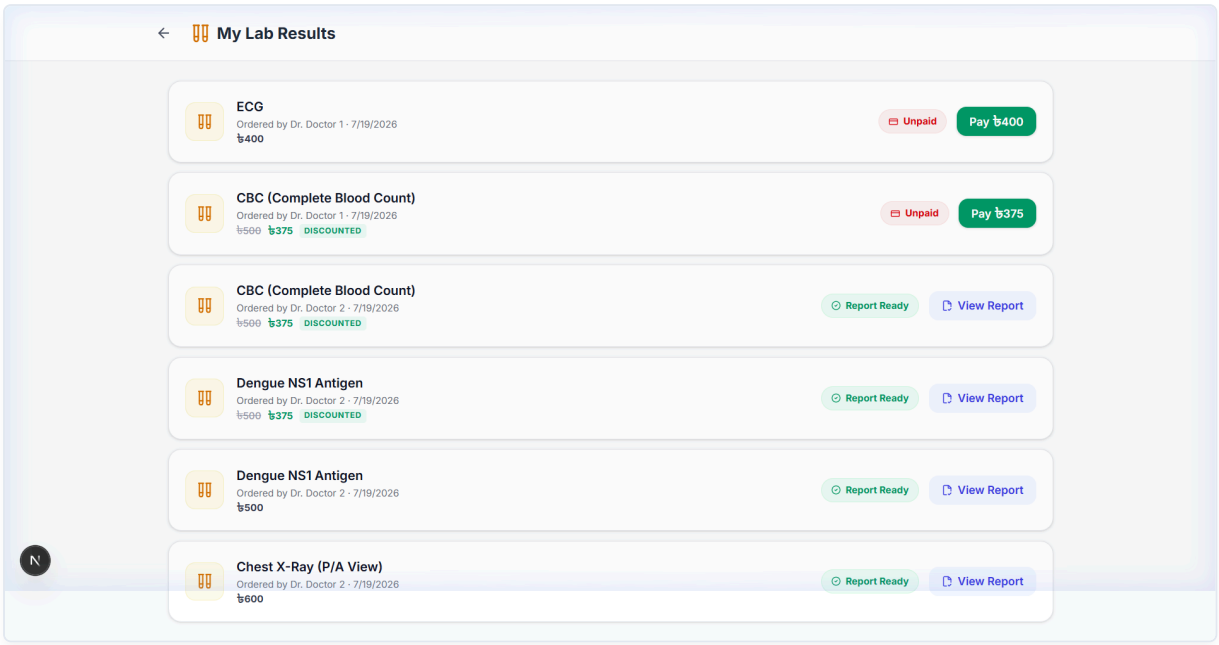


Figure 5: Patient interface for tracking pending lab reports, executing secure payments, and viewing finalized diagnostic findings.

Laboratory Operations Panel

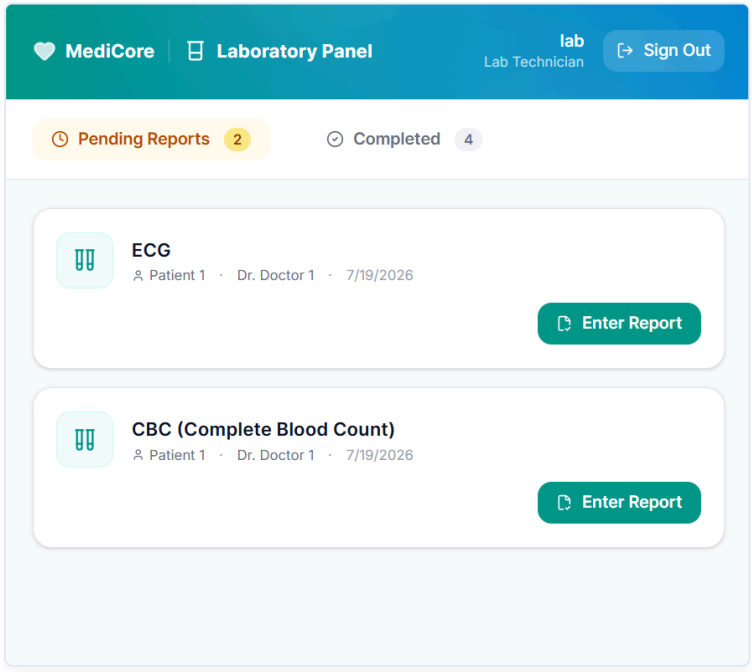


Figure 6: The Lab Technician interface listing paid, pending diagnostic test requests ready for result data entry.

Doctor Management Interfaces

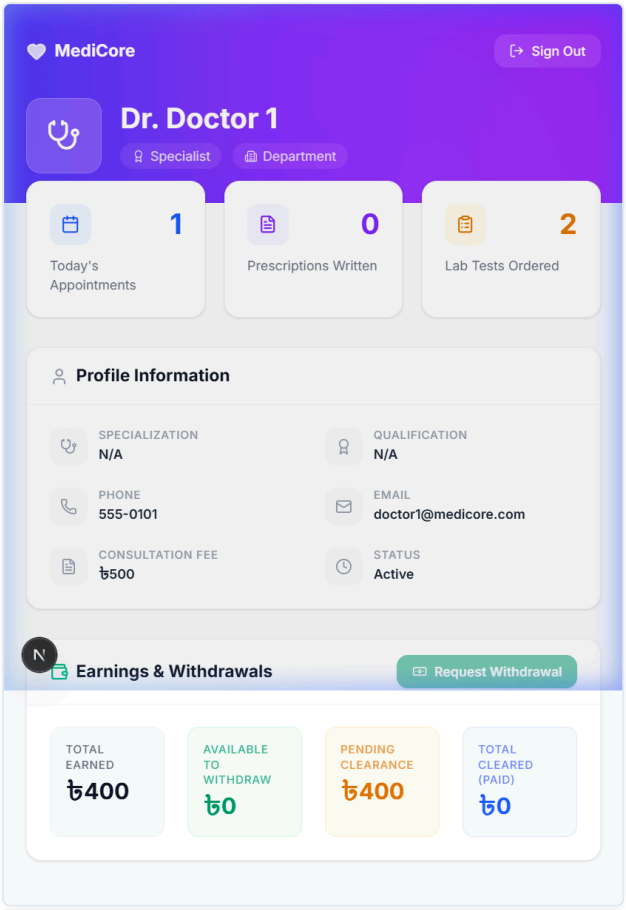


Figure 7: Doctor Profile view highlighting daily appointment queues, generated prescriptions, and a financial earnings withdrawal summary.

7. Conclusion & Future Scope

In conclusion, the MediCore Hospital Management System successfully demonstrates the integration of a modern web stack (Next.js & Express.js) with a robust, enterprise-grade relational database (Oracle 11g). By rigorously applying database normalization principles and utilizing advanced PL/SQL constructs like Sequences and Triggers, the system achieves strict data integrity and concurrency support.

The automated financial ledger and dynamic queue generation algorithms successfully resolve the core problems of manual hospital administration, proving the system's viability as a scalable solution. Future iterations of this project could further enhance the architecture by implementing advanced analytics, mobile application integrations, and AI-driven diagnostic assistance.